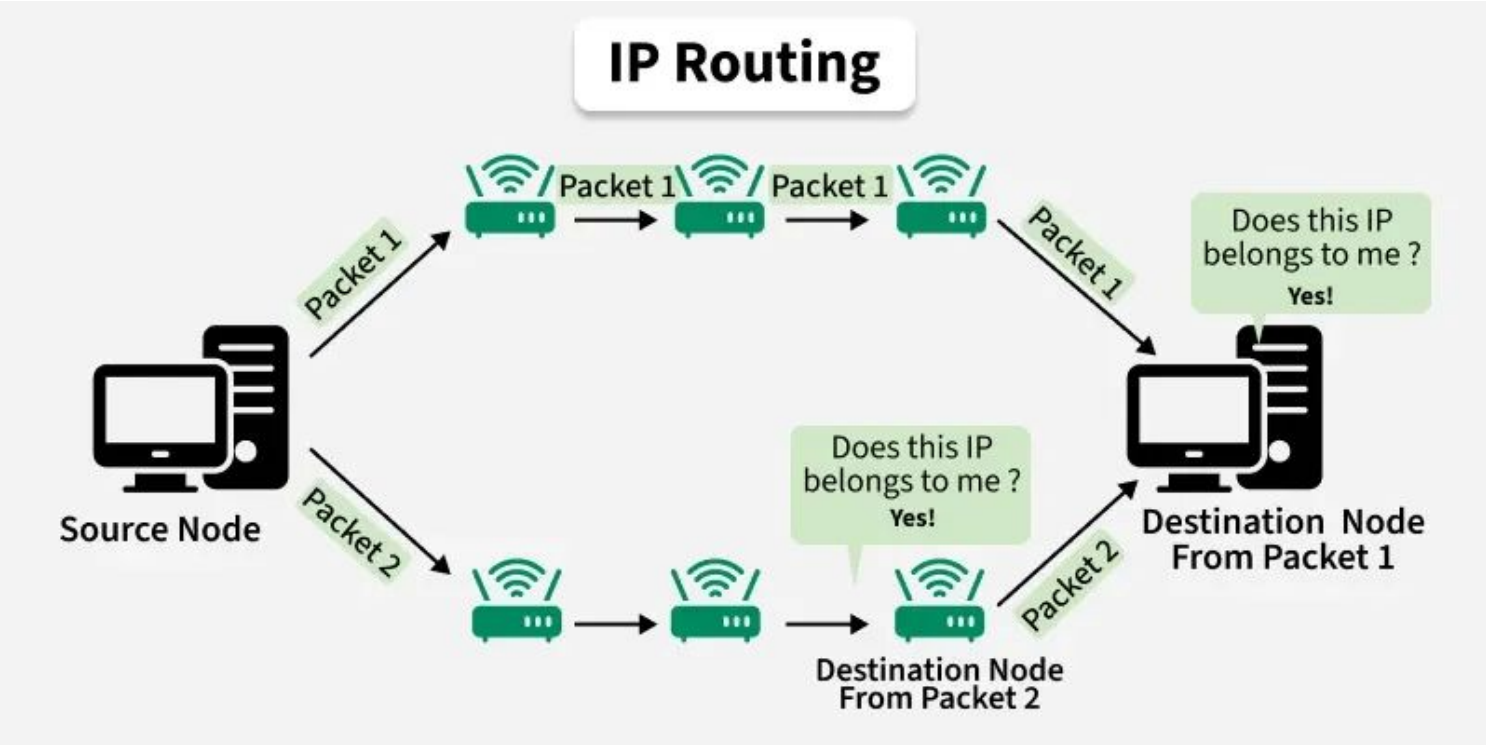
Bellman-Ford Algorithm



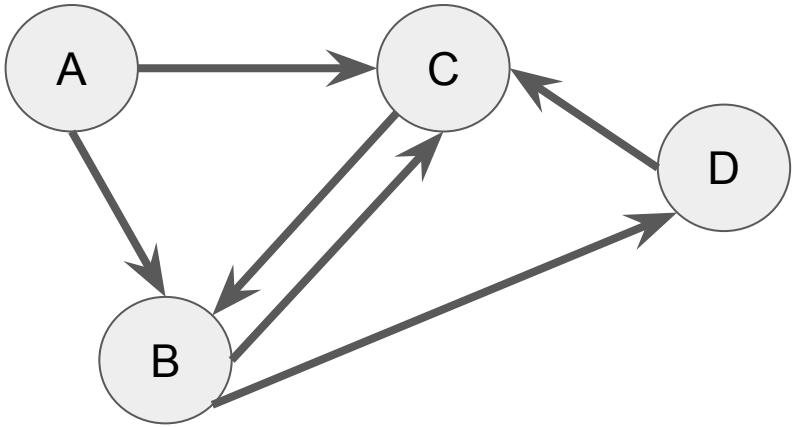
Members:
Vallecera
Valmoria
Yee
Ybañez

Learning Objectives

- Explain Bellman-Ford Algorithm.
- Use cases of Bellman-Ford Algorithm.
- Bellman-Ford Algorithm real world usage.

Bellman-Ford Algorithm

Finding the shortest path from a source vertex to all other vertices in a weighted directed graph.



Bellman-Ford Algorithm 👎

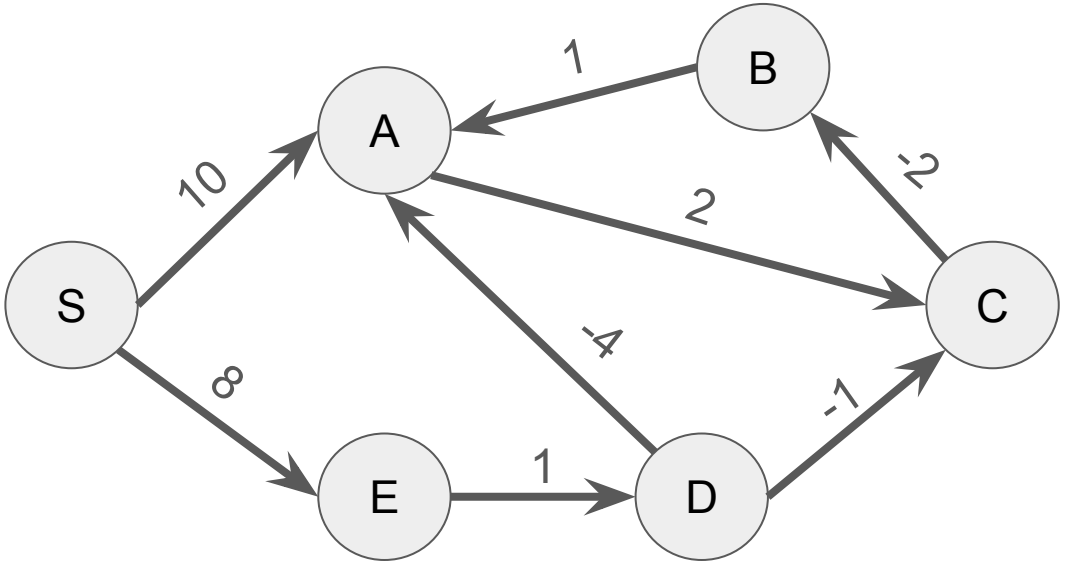
Dijkstra’s Algorithm 👍

Negative weights

Bellman-Ford Algorithm

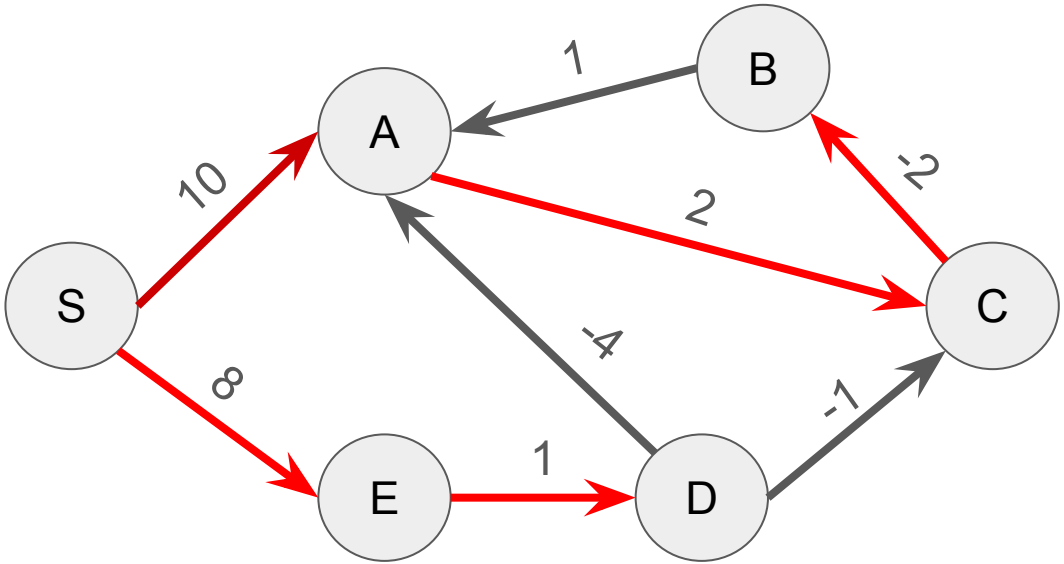
Dijkstra's Algorithm

Illustration:



This is a weighted directed graph. Some of the weights have a negative value.

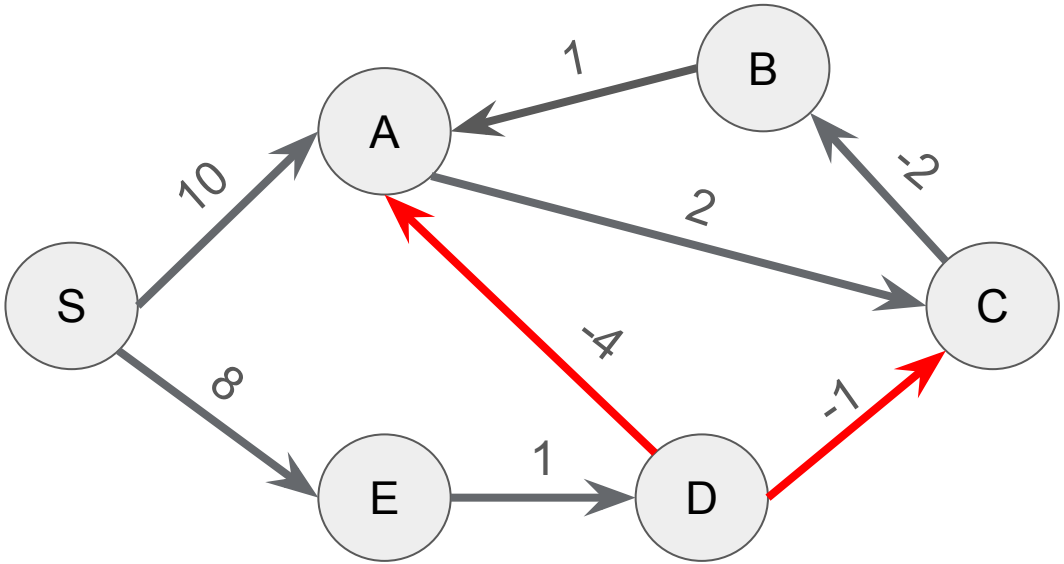
Illustration:



1st iteration ->

S: 0
A: 10
B: 10
C: 12
D: 9
E: 8

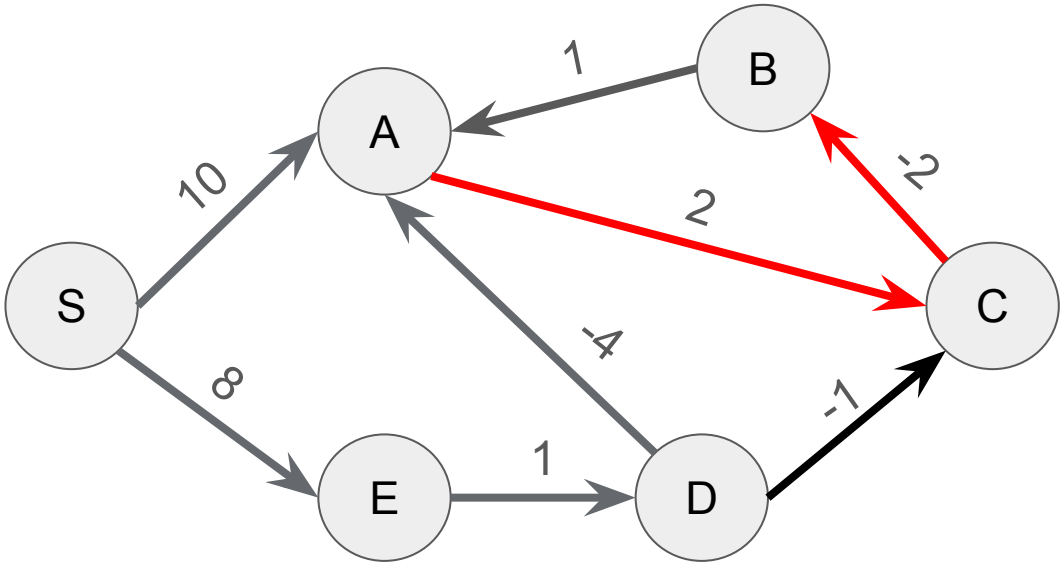
Illustration:



2nd iteration ->

S: 0
A: ~~10~~ - 5
B: 10
C: ~~12~~ - 8
D: 9
E: 8

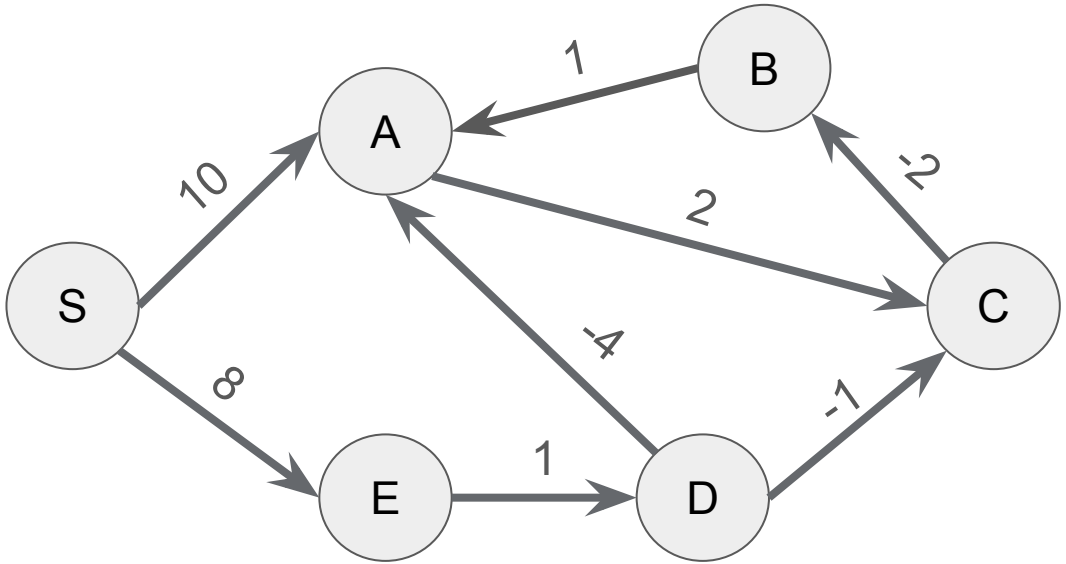
Illustration:



3rd iteration ->

S: 0
A: 5
B: ~~10~~ - 5
C: ~~8~~ - 7
D: 9
E: 8

Illustration:



4th iteration ->

S: 0
A: 5
B: 5
C: 7
D: 9
E: 8

Same value -> you can perform an early stop if nothing changed from the last iteration(improve performance).

Time complexity

Worst and Average

$$O(|V| * |E|)$$

Best

$$O(|E|)$$

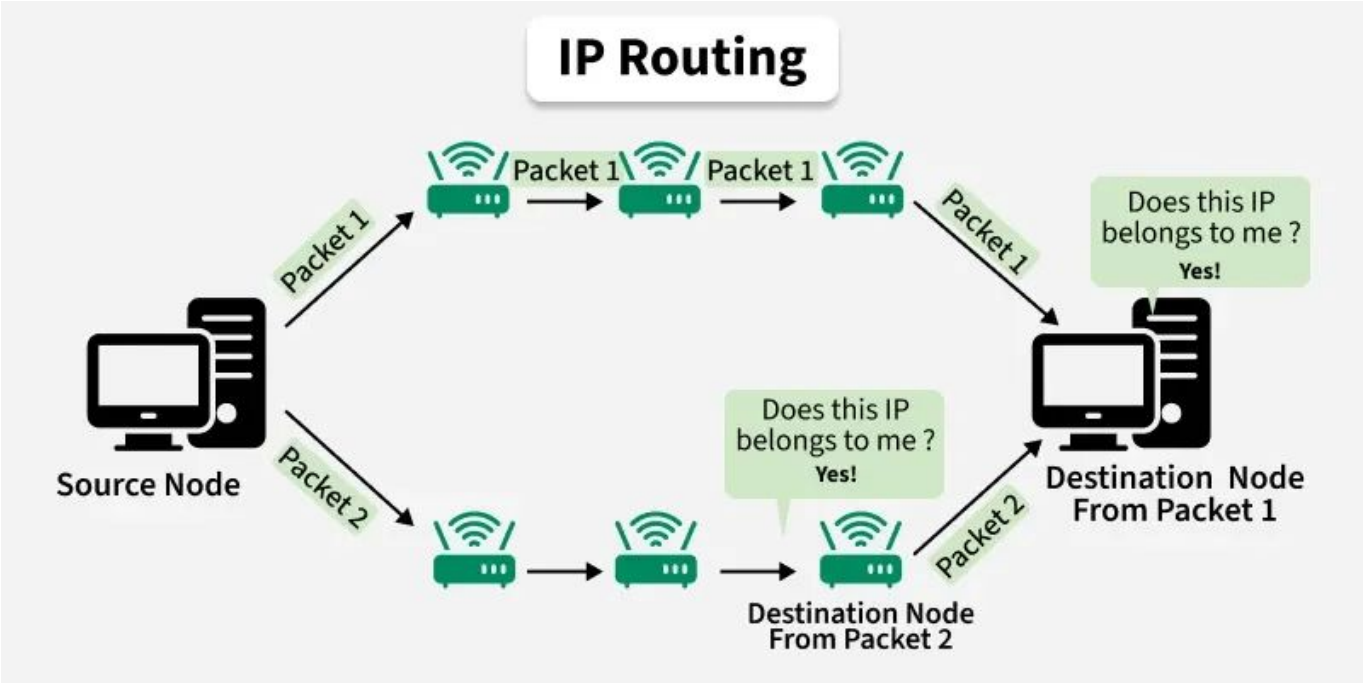
V = Number of vertices/nodes

E = Number of edges

Real World Examples and Applications

Computer Network Routing

Routers calculate the shortest path for data packets based on metrics like delay or congestion, which can fluctuate to represent negative costs.



Logistics & Transportation

For planning routes where some paths might involve subsidies, rebates, or time savings that act as negative costs relative to base costs.

Sample code:

```cpp
#include <iostream>
#include <vector>
using namespace std;

vector<int> bellmanFord(int V, vector<vector<int>>& edges, int src) {

    // Initially distance from source to all
    // other vertices is not known(Infinite).
    vector<int> dist(V, 1e8);
    dist[src] = 0;

    // Relaxation of all the edges V times, not (V - 1) as we
    // need one additional relaxation to detect negative cycle
    for (int i = 0; i < V; i++) {

        for (vector<int> edge : edges) {
            int u = edge[0];
            int v = edge[1];
            int wt = edge[2];
            if (dist[u] != 1e8 && dist[u] + wt < dist[v]) {

                // If this is the Vth relaxation, then there is
                // a negative cycle
                if(i == V - 1)
                    return {-1};

                // Update shortest distance to node v
                dist[v] = dist[u] + wt;
            }
        }
    }

    return dist;
}
```

```cpp
int main() {

    // Number of vertices in the graph
    int V = 5;

    // Edge list representation: {source, destination, weight}
    vector<vector<int>> edges = {
        {1, 3, 2},
        {4, 3, -1},
        {2, 4, 1},
        {1, 2, 1},
        {0, 1, 5}
    };

    // Define the source vertex
    int src = 0;

    // Run Bellman-Ford algorithm to get shortest paths from src
    vector<int> ans = bellmanFord(V, edges, src);

    // Output the shortest distances from src to all vertices
    for (int dist : ans)
        cout << dist << " ";

    return 0;
}
```

Output:

```
0 5 6 6 7
```

Summary

- Very useful for finding the shortest path with negative weights.
- Requires more computations than Dijkstra’s Algorithm.
- Mostly used in graphs with negative edge weights.

Sources

GeeksforGeeks. (2025, July 23). Bellman–Ford Algorithm. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/bellman-ford-algorithm-dp-23/

GeeksforGeeks. (2026, January 21). Routing. GeeksforGeeks. https://www.geeksforgeeks.org/computer-networks/what-is-routing/

Admin. (2024, June 5). What is the Difference Between Logistics and Transportation? Vector Globe. https://vectorglobe.com/difference-between-logistics-and-transportation/